

LAB MANUAL

MCP — the systems tell the model

One lab in five parts: a live Agent Search MCP server, the Bookshelf's own, its resources and prompts, and the identity and audit behind every call

Version 1.0 · September 2026

Environment: Cloud Shell in your sandbox GCP project · Gemini CLI and Gemini 2.5 Flash on Vertex AI · Cloud Run
· the MCP kit (mcp 2.2.0)

Contents

How to use this manual

The Bookshelf behind MCP

Part A — A live server

Part B — Ours

Part C — Resources and prompts

Part D — Identity and audit

Part E — The agent through MCP

Appendix A — settings.json, final

Appendix B — The audit line

Appendix C — The 2026-07-28 revision on one page

Appendix D — The callers, and who is who

Appendix E — The switches

How to use this manual

This is **one lab in five parts**, A to E, run one part at a time right after the chapter of the deck that explains it. Until now you told the model what tools exist: you wrote a `tools.json` and the agent sent it with every request. Today the systems tell it. Part A connects Gemini CLI to a live Agent Search MCP server and asks a policy question through it. Part B deploys the Bookshelf's own MCP server to Cloud Run — its tools defined next to the code that runs them — and calls it from Gemini CLI. Part C reads the server's other two primitives, a resource and a prompt. Part D is identity and audit: the server refuses a call with no token, writes one line per call with your email, and exposes or withholds a tool with one setting. Part E puts the tools behind the agent's own loop and shows two servers in one client. You write no code.

Three programs appear in the printouts. `scripts/mcp-call.py` is a one-shot MCP client for the terminal: it prints what a server says about itself (`server/discover`), its `tools/list`, a resource, a prompt, and one call. Gemini CLI is the interactive one: `/mcp` lists every server in its settings with the tools each published. `agent.py` is the loop with a new switch, `MCP=on`: the tools come from the server's `tools/list` and run there, and its third line says so.

Commands and outputs are in separate blocks

A block captioned **command** holds exactly one line to paste per line shown — the whole line, including any `SWITCH=value` typed in front of `python` or `bash`. A long command soft-wraps on the page; it is still one line.

The block after it, captioned **example** or **expected output**, is what the run prints. Never paste an output block: it goes to the shell and fails with `syntax error near unexpected token`.

Lines you type inside Gemini CLI (`/mcp`, a question) are shown in a block captioned **in Gemini CLI**. Type them at its prompt, not in the shell.

Blocks captioned **example** show one complete printout from a run of the kit, so you can see every line before you read your own. Your service URL is shown as `https://bookshelf-mcp-<...>.a.run.app`; your Google account as `<your_email>`; the model's own words as `<...>`; reservation ids as `<id>`; token counts, costs and timings as `<n>`. The lines that matter — `server/discover`, `tools/list:`, `tools from:`, `⚙️ run`, `caller`, `ok` — are printed as the kit prints them. Where the output is Gemini CLI's or Gemini's, the manual describes its shape in words instead of printing it.

The story for today

Meridian Financial's Bookshelf has a tool-using assistant, already bounded. Its three tools — `lookup_stock`, `search_bookshelf`, `reserve_book` — were defined in a file inside the assistant, `tools.json`, and the caller was a string on the command line. Two things an auditor asks about were in the wrong place: the definition of what the assistant may do lived in the client rather than with the system, and the identity behind a call was whatever the client said it was.

Today the Bookshelf publishes its own tools from a small MCP server that runs next to `bookshelf-api` on Cloud Run. Every client — Gemini CLI, the agent, a one-file script — asks the server what exists and calls it with a Google ID token; the server verifies the token, decides the caller's clearance from the verified email, runs the tool as its own service account, and writes one line per call. Same data store, same API, same three tools — now owned by the system, called with an identity, and written down.

Three staff profiles are used, from `callers.json` (printed in Appendix D): **priya** (cleared for `internal`), **daniel** (`confidential`) and **mlro** (`restricted`). Previously `--user` picked one. Today the server picks one from your verified Google account through the `emails` map in `callers.json`, which the trainer fills in before Part B; an account that is not listed is `priya`. The model never names the caller — the tool has no such argument.

Cost. Part B creates one Cloud Run service (`bookshelf-mcp`, scaled to zero when idle), one service account and one Artifact Registry image; a Cloud Build run of a few minutes. Every Gemini CLI turn and every `agent.py` run is a model call. Nothing else is created; the retrieval resources stay up for the security work that follows.

Set-up (once, before Part A)

Open Cloud Shell in your sandbox project. Upload `regai-d4-mcp-kit.zip` (the upload button in the Cloud Shell toolbar), unzip it, activate the venv, install the packages the kit needs, and copy the two patched files over the agent kit — they replace `agent.py` and `tools_impl.py` there, and everything the agent kit already does still works (`bash scripts/selftest.sh` in the agent kit passes as before).

COMMAND — PASTE THIS LINE
`unzip regai-d4-mcp-kit.zip`

COMMAND — PASTE THIS LINE
`source ~/regai-venv/bin/activate && pip install -q mcp google-auth requests`

COMMAND — PASTE THIS LINE
`cp regai-d4-mcp-kit/agent-patch/*.py regai-d2-agent-kit/`

Gemini CLI: Cloud Shell may already have it. The first command prints its version if it is there and installs it if not.

COMMAND — PASTE THIS LINE
`gemini --version || npm install -g @google/gemini-cli`

EXPECTED OUTPUT
 <the version number, e.g. 0.x.y — or npm's install progress, then nothing more>

Gemini CLI reaches Gemini through Vertex AI in your project, like `agent.py`. Three variables tell it so; set them in every new Cloud Shell tab you open today.

COMMAND — PASTE THIS LINE
`export GOOGLE_GENAI_USE_VERTEXAI=true GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)`

COMMAND — PASTE THIS LINE
`export GOOGLE_CLOUD_LOCATION=us-central1`

Then one check that the venv can import the MCP client library and that the kit's client script runs. `--help` prints the script's own usage text.

COMMAND — PASTE THIS LINE
`cd regai-d4-mcp-kit && python scripts/mcp-call.py --help`

EXPECTED OUTPUT

```
usage: mcp-call.py [-h] --url URL [--token TOKEN] [--header NAME: VALUE]
                  [--list] [--call NAME JSON] [--resources] [--read URI]
                  [--prompts] [--get NAME JSON]

mcp-call.py — one-shot MCP client: see tools/list and call one tool, from a
terminal.

options:
  -h, --help            show this help message and exit
  --url URL              the MCP endpoint, e.g. https://.../mcp
  --token TOKEN          auto | none | <token>
  --header NAME: VALUE  an extra request header (repeatable)
  --list                print tools/list
  --call NAME JSON       call one tool
  --resources            print resources/list
  --read URI             read one resource by uri
  --prompts             print prompts/list
  --get NAME JSON        get one prompt
```

The whole lab, in order — every step is explained in its part:

THE ORDER

Part A scripts/gemini-config.sh · gemini, /mcp, the three Agent Search tools · the question scripts/mcp-call.py --list against the server: the raw tools/list on the wire
Part B bookshelf-mcp/deploy.sh · verify.sh · gemini-config.sh · /mcp: bookshelf, two tools a stock question and a reserve request in Gemini CLI
Part C mcp-call.py --resources · --read · --prompts · --get: the resource and the prompt
Part D mcp-call.py with no token, then with yours · audit-log.sh · EXPOSE_RESERVE=on · the fence
Part E the agent with MCP=on · both servers in one Gemini CLI · (optional) APPROVE=ask

THE LAB · DIFFICULTY ●●○

The Bookshelf behind MCP

Objective

Put the Bookshelf's tools behind an MCP server the system owns, and call them with an identity. By the end you have: Gemini CLI connected to a live Agent Search MCP server and a policy question answered through a tool nobody in the room defined; the Bookshelf's own MCP server deployed to Cloud Run, listed in Gemini CLI with the two tools it chose to publish; its resource and its prompt read from the terminal; a request with no token refused and a request with yours answered; the server's audit log with your email on every call; the withheld `reserve_book` exposed with one setting and fenced off again on the client; the agent driven with `tools.json` never read; and two servers in one client at once.

Environment

Cloud Shell in your own sandbox project, with the venv (`~/regai-venv`) active, the agent kit (`regai-d2-agent-kit`) next to this kit (`regai-d4-mcp-kit`), and Gemini CLI installed. Parts A, C and D run from this kit's folder; Part B deploys a Cloud Run service from it; Part E runs the agent from the agent kit's folder. The Agent Search data store and app, and the `bookshelf-api` service on Cloud Run, must be up.

Prerequisites

- The Agent Search data store and app are up; the `bookshelf-api` service is deployed on Cloud Run
- The agent kit folder (`regai-d2-agent-kit`) is present, with its `agent.env` filled in and `TOOLS=on`
- The set-up above done: this kit unzipped, packages installed, `agent-patch` copied, Gemini CLI present, the three Vertex variables exported
- The deck's chapter for each part taught before you start that part — each part assumes its chapter

The five parts

A — A live server. `gemini-config.sh` writes `agent-search` into Gemini CLI's settings. `/mcp` shows three tools with the schemas the server published; the question is answered through one of them. Then the raw `tools/list` from the terminal.

B — Ours. `deploy.sh` puts `bookshelf-mcp` on Cloud Run; `verify.sh` calls it; `/mcp` shows two tools; a stock question is answered through MCP and a reserve request finds no tool.

C — Resources and prompts. `mcp-call.py --resources, --read, --prompts, --get`: the catalogue resource and the `policy_lookup` prompt — the two primitives beyond tools.

D — Identity and audit. No token: refused. Your token: the tools. `audit-log.sh`: your calls, with your email. `EXPOSE_RESERVE=on`: a third tool, a reservation, and the log says who. `INCLUDE_TOOLS`: the client's fence.

E — The agent through MCP. `MCP=on: tools from: ...`, `tools.json` never read. Then both servers in one Gemini CLI, and — if there is time — the approval card in front of a server's tool.

Part A — A live server

Goal

Connect Gemini CLI to a live Agent Search MCP server with your own Google credentials, read the three tools it publishes — `search`, `conversational_search`, `list_engines` — and their schemas, and ask a policy question naming your Agent Search engine. The answer comes through MCP: Gemini CLI calls a tool it discovered, the server searches the approved-documents data store, and the cited answer comes back. Then see the same `tools/list` from the terminal, raw — the protocol on the wire.

Steps

1. From the kit folder, write the two servers into `~/.gemini/settings.json`. Only `agent-search` is written now: `bookshelf` needs a deployed service, which Part B creates. Other settings in the file are kept.

COMMAND — PASTE THIS LINE

```
cd ~/regai-d4-mcp-kit
```

COMMAND — PASTE THIS LINE

```
bash scripts/gemini-config.sh
```

EXPECTED OUTPUT

```
wrote /home/<you>/.gemini/settings.json
agent-search: https://discoveryengine.googleapis.com/mcp
(bookshelf not added: no bookshelf-mcp service found yet – run this again after bookshelf-mcp/deploy.sh)
Start Gemini CLI with: gemini then type /mcp to see both servers and their tools.
```

What it wrote is `lab-parts/part-a.settings.json` with your project id filled in: the server's `httpUrl`, `authProviderType: google_credentials` with the `cloud-platform` scope (Gemini CLI presents your own Google credentials), and an `X-Goog-User-Project` header naming your project.

2. Start Gemini CLI and list the servers. The first start may ask you to choose an authentication method — pick Vertex AI, which the three exported variables configure.

COMMAND — PASTE THIS LINE

```
gemini
```

IN GEMINI CLI

```
/mcp
```

`/mcp` lists `agent-search` as connected, and under it the three tools — `search`, `conversational_search`, `list_engines` — each with its description and the parameters the server published. Read the schemas on screen: which tool takes an engine or app name, which takes a query, what is required. Nobody wrote a `tools.json` for these; Gemini CLI sent `tools/list` and this is what came back. Write down the three names and the parameter each needs for a search.

3. Ask the policy question, naming your Agent Search engine. One line at the Gemini CLI prompt.

IN GEMINI CLI

```
Using the bookshelf-app engine, which form must a cardholder sign before a dispute can go to chargeback?
```

Gemini CLI shows the tool it wants to call, with its arguments, and asks you to allow it (the server is not marked trusted, so it asks per call) — say yes. The server searches the approved-documents data store with your credentials and the answer comes back with citations; the model reports it. Which of the three tools it chose, and with what arguments, is the model's decision: write down what yours did. The form named should be the one the store holds — `FRM-CD-7` from the dispute policy — if your account may see confidential documents through the engine.

4. Leave Gemini CLI (/quit). The same `tools/list`, from the terminal, with an access token and the project header. This is what Gemini CLI did before `/mcp` could show anything.

COMMAND — PASTE THIS LINE

```
P=$(gcloud config get-value project)
```

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url https://discoveryengine.googleapis.com/mcp --token "$(gcloud auth print-access-token)"
--header "X-Goog-User-Project: $P" --list
```

EXPECTED OUTPUT — SHAPE

```
url: https://discoveryengine.googleapis.com/mcp token: sent
server/discover: <versions and cache hints – or "not supported – initialize handshake, protocol <...>" if the
  server negotiates the older way>
server: <the server's name and version>
capabilities: {<what it declares>}
tools/list: 3 tools< ttlMs=<n> cacheScope=<...>, when the server sends them>
- search <annotations, if any>
  <its description>
  <its parameters, * = required>
- conversational_search ...
- list_engines ...
```

Read the first line — what the server says about itself — and then `tools/list:`. Three tools, in a fixed order; `ttlMs` and `cacheScope` when the server sends them; each tool's description and parameters, * marking the required ones. That is the 2026-07-28 revision on the wire: no session to set up, a list a client may cache, one deterministic order. The next part runs the same three lines against a server you deploy.

✓ Check yourself

`gemini-config.sh` wrote `agent-search` and said why `bookshelf` was not added yet.

`/mcp` showed `agent-search` connected with `search`, `conversational_search`, `list_engines` and their schemas.

The question was answered after Gemini CLI called one of the three tools — you wrote down which, and with what arguments.

`mcp-call.py --list` printed `tools/list: 3 tools` and the same three names, from the terminal.

Troubleshooting

Symptom	Cause	Fix
No project: gcloud config set project <id>	gcloud has no default project in this tab	<code>gcloud config set project <your-project-id></code> , rerun
<code>/mcp</code> shows <code>agent-search</code> as disconnected	No Google credentials in this Cloud Shell session, or the Discovery Engine API is not enabled	<code>gcloud auth login</code> ; enable the API for the data store; restart <code>gemini</code>
Gemini CLI does not use Vertex AI (asks for an API key)	The three variables are not set in this tab	Export them again (set-up), restart <code>gemini</code>
The answer does not name the form, or says nothing was found	The engine name in the question does not match your app, or your account's view of the store	Use the app's id from the console in the question; ask the trainer which account sees confidential documents
<code>mcp-call.py</code> prints <code>http: 403</code>	<code>X-Goog-User-Project</code> missing or naming a project your account cannot use	Check <code>P</code> ; the header is the project <code>id</code>
<code>ModuleNotFoundError: No module named 'mcp'</code>	The venv is not active, or <code>pip install</code> was skipped	<code>source ~/regai-venv/bin/activate</code> && <code>pip install -q mcp google-auth requests</code>

Part B — Ours

Goal

Deploy the Bookshelf's MCP server to Cloud Run and call it from Gemini CLI. `deploy.sh` builds the image and deploys `bookshelf-mcp` as its own service account, IAM-protected — start it and keep listening; it takes a few minutes. `verify.sh` lists the tools and calls one with your ID token, then is refused without. Gemini CLI: `/mcp` shows `bookshelf` with the two tools `server.py` chose to publish; a stock question is answered through MCP; a reserve request finds no tool.

Steps

1. Deploy. The script enables the APIs, creates the service account `bookshelf-mcp-sa`, grants it `roles/run.invoker` on `bookshelf-api` plus `roles/aiplatform.user` and `roles/discoveryengine.viewer`, builds the image with Cloud Build, and deploys with `--no-allow-unauthenticated`. It is idempotent: if it fails, fix the cause and run it again.

COMMAND — PASTE THIS LINE

```
bash bookshelf-mcp/deploy.sh
```

EXPECTED OUTPUT — THE SCRIPT'S OWN LINES; GLOUD'S PROGRESS OMITTED

```
project      : <your-project-id>
region       : asia-south1
bookshelf-api: https://bookshelf-api-<...>.a.run.app

1/6 enabling APIs...
2/6 service account bookshelf-mcp-sa...
3/6 roles: invoke bookshelf-api, use Vertex AI and Discovery Engine...
4/6 Artifact Registry repository...
5/6 building the image with Cloud Build...
<Cloud Build's progress, a few minutes>
6/6 deploying to Cloud Run...
<Cloud Run's progress>

=====
bookshelf-mcp is deployed
MCP endpoint: https://bookshelf-mcp-<...>.a.run.app/mcp
runs as      : bookshelf-mcp-sa@<your-project-id>.iam.gserviceaccount.com

Save it for the labs:
export BOOKSHELF_MCP_URL=https://bookshelf-mcp-<...>.a.run.app
Then: bash verify.sh
=====
```

2. Paste the `export` line the script printed — every later command uses `$BOOKSHELF_MCP_URL`. Then `verify: tools/list` and one call with your ID token, then `tools/list` without one.

COMMAND — PASTE THE LINE `DEPLOY.SH` PRINTED, WITH YOUR URL

```
export BOOKSHELF_MCP_URL=https://bookshelf-mcp-<...>.a.run.app
```

COMMAND — PASTE THIS LINE

```
bash bookshelf-mcp/verify.sh
```

EXAMPLE — THE TWO LISTINGS ARE THE KIT'S; THE REFUSAL IS CLOUD RUN'S

```
endpoint : https://bookshelf-mcp-<...>.a.run.app/mcp
as       : <your email>

--- tools/list (with your ID token)
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: sent
server/discover: versions=['2026-07-28'] ttlMs=0 cacheScope=private
server: bookshelf 1.0
instructions: The Meridian Bookshelf library for bank staff: stock, reservations and answers from the
  approved policies and procedures.
capabilities: {"prompts": {"listChanged": true}, "resources": {"subscribe": true, "listChanged": true},
  "tools": {"listChanged": true}}
tools/list: 2 tools ttlMs=300000 cacheScope=public
- lookup_stock {"readOnlyHint": true}
  Read-only: how many copies of one document in the Bookshelf library are
  available right now. Use for questions about stock or copies left; needs the
  document id. Changes nothing.
```

```

    book_id*: {"type": "string"} Document id, e.g. PRC-KYC-003
- search_bookshelf {"readOnlyHint": true}
  Search Meridian's approved policies and procedures and answer a question from
  them, with citations. Use for questions about rules, forms, limits or procedures.
  The caller's clearance, taken from their verified identity, decides which documents
  are searched. Changes nothing.
  question*: {"type": "string"} The question, in the caller's words

--- tools/call lookup_stock
tools/call lookup_stock({"book_id": "PRC-KYC-003"})
→ {"book_id": "PRC-KYC-003", "title": "Customer Identification and Verification Procedure",
    "copies_available": 3, "copies_total": 3}

--- the same tools/list with no token (expect a refusal: 403 from Cloud Run IAM)
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: none
error: <the client's error>
http: 403 Forbidden <the start of Cloud Run's HTML error page>

done — the server answers with a token and refuses without one.
Your email is now in the audit log: bash ../scripts/audit-log.sh 5

```

Read the first listing line by line. `server/discover: versions=['2026-07-28']` — the server speaks the current revision and needs no session. `server: bookshelf 1.0` and its instructions. `tools/list: 2` tools `ttlMs=300000` `cacheScope=public` — two tools, cacheable for five minutes by anyone. Then each tool: its annotation (`readOnlyHint: true`), the description from `server.py`'s docstring, and its one parameter with * for required. `reserve_book` is not there. The call returns a `copies_available` count from `bookshelf-api`, reached by the server as `bookshelf-mcp-sa`, not as you. The refusal at the end is Cloud Run's IAM check, which answers before the container sees the request; the server's own check is Part D.

3. Add the server to Gemini CLI. The script now finds `bookshelf-mcp` and writes `bookshelf` with your fresh ID token in the `Authorization` header — the token lasts about an hour; when Gemini CLI reports a 401 from `bookshelf`, run this script again.

COMMAND — PASTE THIS LINE

```
bash scripts/gemini-config.sh
```

EXPECTED OUTPUT

```

wrote /home/<you>/.gemini/settings.json
  agent-search: https://discoveryengine.googleapis.com/mcp
  bookshelf: https://bookshelf-mcp-<...>.a.run.app/mcp
Start Gemini CLI with: gemini then type /mcp to see both servers and their tools.

```

4. Start Gemini CLI and look. `bookshelf` shows two tools, `lookup_stock` and `search_bookshelf`, each with the description and schema `server.py` declared — the same text as the `mcp-call.py` listing above.

COMMAND — PASTE THIS LINE

```
gemini
```

IN GEMINI CLI

```
/mcp
```

5. Ask a stock question, then ask it to reserve.

IN GEMINI CLI

```
How many copies of PRC-KYC-003 are left?
```

Gemini CLI asks to run `lookup_stock` on `bookshelf` with `book_id: PRC-KYC-003` — allow it. Our server verifies your token, calls `bookshelf-api` as `bookshelf-mcp-sa`, and returns the count; the model reports it (3 of 3 if the library was reset; `python agent.py --reset` in the agent kit puts it back).

IN GEMINI CLI

```
Reserve one copy of PRC-KYC-003 for daniel
```

There is no tool for it, and the model says so in its own words. It is not refusing: it has nothing to call. The server withheld `reserve_book`, and no prompt on this side can bring it back — Part D flips the setting that can. Leave Gemini CLI with `/quit`.



`deploy.sh` ended with the box: the MCP endpoint and runs as `bookshelf-mcp-sa@...`
`BOOKSHELF_MCP_URL` is exported in this tab.

`verify.sh: tools/list: 2 tools ttlMs=300000 cacheScope=public`, a `lookup_stock` result,
and a refusal without a token.

`/mcp: bookshelf` with `lookup_stock` and `search_bookshelf`; the stock question answered; the
reserve request found no tool.

Troubleshooting

Symptom	Cause	Fix
<code>deploy.sh</code> fails in step 5 (Cloud Build)	Your account lacks <code>roles/cloudbuild.builds.editor</code> , or the API is not enabled	Ask the project owner for the role, or enable Cloud Build in the console; rerun — it is idempotent
<code>deploy.sh</code> : unauthenticated access refused by organisation policy	The org forbids public Cloud Run services	Nothing to change: the service is deployed <code>--no-allow-unauthenticated</code> ; every caller presents an ID token
<code>bookshelf-api</code> not found in <code><project>/asia-south1</code>	That service is in another project or region	Deploy <code>bookshelf-api</code> first, or <code>export BOOKSHELF_API_URL=...</code> and rerun
<code>verify.sh</code> or <code>mcp-call.py</code> prints <code>http: 403</code> with your token	Your account lacks <code>roles/run.invoker</code> on <code>bookshelf-mcp</code>	<code>gcloud run services add-iam-policy-binding bookshelf-mcp --member="user:\$(gcloud config get-value account)" --role=roles/run.invoker --region=asia-south1</code>
<code>mcp-call.py</code> prints <code>error: 401</code>	The token reached the server and did not verify — usually expired	Run again: <code>--token auto</code> fetches a fresh one
Gemini CLI: <code>bookshelf</code> disconnected, or a 401	The token in <code>settings.json</code> has expired (about an hour)	<code>bash scripts/gemini-config.sh</code> again, restart <code>gemini</code>
HTTP 409 ... only <code>n</code> copies available	The library is low	<code>python agent.py --reset</code> in the agent kit

Part C — Resources and prompts

Goal

Read the two primitives beyond tools. The server publishes a **resource** — read-only context a client fetches by URI, here `bookshelf://catalogue`, the library's index — and a **prompt** — a template the server owns, `policy_lookup.mcp-call.py --resources` and `--read` show the resource with its cache hints; `--prompts` and `--get` show the prompt and fill it. For each, say who triggers it: the model, the client, or the user.

Steps

1. The server is deployed from Part B. List its resources, then read the one it has. `--read` takes the resource's URI.

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --resources
```

EXAMPLE

```
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: sent
server/discover: versions=['2026-07-28'] ttlMs=0 cacheScope=private
server: bookshelf 1.0
capabilities: {"prompts": ..., "resources": ..., "tools": {"listChanged": true}}
resources/list: 1 resources ttlMs=300000 cacheScope=public
- bookshelf://catalogue (catalogue) application/json
  The index of Meridian policy and procedure documents: id, title, classification and status. Read-only
  reference, not document text.
```

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --read bookshelf://catalogue
```

EXAMPLE — TRIMMED; THE RUN PRINTS ALL 13

```
resources/read bookshelf://catalogue ttlMs=300000 cacheScope=public
{
  "documents": [
    {
      "id": "HB-BLB-001",
      "title": "Buy Local Bonds — Product Handbook",
      "classification": "restricted",
      "status": "approved"
    },
    ... 12 more documents, one per policy or procedure ...
  ]
}
```

`resources/list` carries `ttlMs=300000 cacheScope=public` — the same caching contract as `tools/list`; a client may hold the catalogue for five minutes and share it. `resources/read` returns the library's **index** — id, title, classification, status — not the text of any policy. A resource is triggered by the client application, not the model: the application decides to put this reference in front of the model, because you do not want the model deciding whether to see what documents exist.

2. List the prompts, then fill one. `--get` takes the prompt's name and a JSON object of its arguments.

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --prompts
```

EXAMPLE

```
prompts/list: 1 prompts
- policy_lookup
  Frame a staff question about a Meridian policy or procedure so the answer stays grounded in the approved
  documents, with citations.
  topic*:
```

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --get policy_lookup '{"topic": "card disputes"}'
```

EXAMPLE

```
prompts/get policy_lookup
[user] Using the Bookshelf, find Meridian's approved guidance on: card disputes. Answer only from the
      documents the caller is cleared to see, quote the document id for each point, and say plainly if nothing
      approved covers it.
```

`policy_lookup` has one argument, `topic` (the `*` marks it required). `--get` fills it into a `[user]` message. The wording of the message lives on the **server**, not with the person at the keyboard: every client that uses `policy_lookup` frames a policy question the same way, and improving that wording is a redeploy, not a memo. A prompt is triggered by the user — they pick it from the client's menu.

3. See both in Gemini CLI. `/mcp` now lists the resource and the prompt in their own sections beside the two tools.

COMMAND — PASTE THIS LINE

```
gemini
```

IN GEMINI CLI

```
/mcp
```

Under `bookshelf`, Gemini CLI shows the two tools, the `catalogue` resource and the `policy_lookup` prompt. `/quit` when you have seen them.

✓ Check yourself

`--resources` printed `resources/list: 1 resources` with the cache hints, and `bookshelf://catalogue`.

`--read bookshelf://catalogue` returned the index (id, title, classification, status) — not document text.

`--prompts` printed `policy_lookup` and its required `topic` argument; `--get` returned the filled `[user]` message.

You can say, for each primitive, who triggers it: the model (tool), the client (resource), the user (prompt).

Troubleshooting

Symptom	Cause	Fix
<code>--read</code> prints error: resource not found	The URI is mistyped	Copy it from <code>--resources: bookshelf://catalogue</code>
<code>--get</code> prints error: missing argument topic	No JSON object, or the key is wrong	<code>--get policy_lookup '{"topic": "..."}'</code> — the argument name is <code>topic</code>

Symptom	Cause	Fix
<code>/mcp</code> does not show resources or prompts sections	The token in <code>settings.json</code> expired, or an older Gemini CLI	<code>bash scripts/gemini-config.sh</code> , restart <code>gemini</code> ; the terminal <code>mcp-call.py</code> shows them regardless
<code>http: 403</code> with your token	<code>roles/run.invoker</code> on <code>bookshelf-mcp</code> is missing	The one-line grant in Part B's troubleshooting

Part D — Identity and audit

Goal

Who is calling, and what was done. A request with no token is refused; with your token the tools are listed. `audit-log.sh` reads the server's log back as a table: your earlier calls, with your email. Then expose the withheld tool on the server with one setting — `EXPOSE_RESERVE=on` — and the list has three; reserve one copy from Gemini CLI and the log says who. Then fence it on the client: `INCLUDE_TOOLS` makes `/mcp` show two tools while the server still has three. Two fences, two owners.

Steps

1. Ask for the list with no token at all, then with yours (`--token auto`, the default, sends `gcloud auth print-identity-token`).

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --token none --list
```

EXAMPLE — THE SERVER'S OWN CHECK, ON A LOCAL RUN

```
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: none
error: 401: no valid Google ID token (no bearer token) (code -32001)
http: 401 Unauthorized {"jsonrpc":"2.0","id":null,"error":{"code":-32001,"message":"401: no valid Google ID token (no bearer token)"}}
```

On Cloud Run the first refusal you see is Cloud Run's: `http: 403 Forbidden` and the start of an HTML page, because the IAM check answers before the container is reached. The example above is what the server itself answers when a request with no token does reach it — `401`, a JSON-RPC error with the reason, and an audit line with caller `anonymous` (`scripts/selftest.sh` runs that case locally). Two refusals, two layers: IAM, then the code in `server.py`, which checks the signature, the issuer (`accounts.google.com`), the audience and the expiry with `google.oauth2.id_token`, and takes the `email` claim as the caller.

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --list
```

EXAMPLE

```
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: sent
server/discover: versions=['2026-07-28'] ttlMs=0 cacheScope=private
server: bookshelf 1.0
instructions: The Meridian Bookshelf library for bank staff: stock, reservations and answers from the
approved policies and procedures.
capabilities: {"prompts": {"listChanged": true}, "resources": {"subscribe": true, "listChanged": true},
"tools": {"listChanged": true}}
tools/list: 2 tools ttlMs=300000 cacheScope=public
- lookup_stock {"readOnlyHint": true}
  Read-only: how many copies of one document in the Bookshelf library are
  available right now. Use for questions about stock or copies left; needs the
  document id. Changes nothing.
  book_id*: {"type": "string"} Document id, e.g. PRC-KYC-003
- search_bookshelf {"readOnlyHint": true}
  Search Meridian's approved policies and procedures and answer a question from
  them, with citations. Use for questions about rules, forms, limits or procedures.
  The caller's clearance, taken from their verified identity, decides which documents
  are searched. Changes nothing.
  question*: {"type": "string"} The question, in the caller's words
```

2. What was done. Every call is one JSON line on the server's stdout, which Cloud Logging stores as a structured entry; `audit-log.sh` reads the last N back as a table. New entries take a few seconds to appear.

COMMAND — PASTE THIS LINE

```
bash scripts/audit-log.sh 20
```

EXPECTED OUTPUT — ONE ROW PER CALL SO FAR, OLDEST FIRST

time (UTC)	caller	tool	ok	ms	args
<date> <time>	<your email>	lookup_stock	ok	<n>	{"book_id": "PRC-KYC-003"}
<date> <time>	<your email>	lookup_stock	ok	<n>	{"book_id": "PRC-KYC-003"}
<date> <time>	<your email>	search_bookshelf	ok	<n>	{"question": "..."}

Your `verify.sh` call and the Gemini CLI calls are there, each with your email — the identity the server verified, not a `--user` you typed. Any call that failed shows **FAIL** with the error text. The columns are the audit line's fields (Appendix B).

3. Expose the withheld tool on the server. One environment variable, a new revision, a minute or so; then the list again.

COMMAND — PASTE THIS LINE

```
gcloud run services update bookshelf-mcp --region asia-south1 --update-env-vars EXPOSE_RESERVE=on
```

COMMAND — PASTE THIS LINE

```
python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --list
```

EXAMPLE

```
url: https://bookshelf-mcp-<...>.a.run.app/mcp token: sent
server/discover: versions=['2026-07-28'] ttlMs=0 cacheScope=private
server: bookshelf 1.0
instructions: The Meridian Bookshelf library for bank staff: stock, reservations and answers from the
approved policies and procedures.
capabilities: {"prompts": {"listChanged": true}, "resources": {"subscribe": true, "listChanged": true},
"tools": {"listChanged": true}}
tools/list: 3 tools ttlMs=300000 cacheScope=public
- lookup_stock {"readOnlyHint": true}
  Read-only: how many copies of one document in the Bookshelf library are
  available right now. Use for questions about stock or copies left; needs the
  document id. Changes nothing.
  book_id*: {"type": "string"} Document id, e.g. PRC-KYC-003
- search_bookshelf {"readOnlyHint": true}
  Search Meridian's approved policies and procedures and answer a question from
  them, with citations. Use for questions about rules, forms, limits or procedures.
  The caller's clearance, taken from their verified identity, decides which documents
  are searched. Changes nothing.
  question*: {"type": "string"} The question, in the caller's words
- reserve_book {"readOnlyHint": false, "destructiveHint": false, "idempotentHint": false}
  Reserve 1 to 3 copies of a document for the calling member of staff. Use only
  when the caller asks to reserve, hold or book copies. This changes library records:
  stock goes down and a reservation is created in the caller's name.
  book_id*: {"type": "string"} Document id, e.g. PRC-KYC-003
  copies*: {"maximum": 3, "minimum": 1, "type": "integer"} Copies to reserve, 1 to 3
```

Three tools now, and read `reserve_book`'s annotations: `readOnlyHint: false`, `destructiveHint: false`, `idempotentHint: false` — the server tells every client that this one changes records. Its two parameters carry the schema from the type hints: `copies` is an integer from 1 to 3.

- Reserve one copy from Gemini CLI, and see who did it. Gemini CLI reads a server's tool list when it starts, so start it fresh; if `/mcp` still shows two tools, run `bash scripts/gemini-config.sh` and start it again.

COMMAND — PASTE THIS LINE
gemini

IN GEMINI CLI
/mcp

IN GEMINI CLI
Reserve one copy of PRC-KYC-003 for daniel

`/mcp` shows `reserve_book` under `bookshelf`. The reserve request now finds a tool: Gemini CLI asks to run `reserve_book` with `book_id: PRC-KYC-003`, `copies: 1` — allow it — and the model reports the reservation id and the copies left. `/quit`, then the log.

COMMAND — PASTE THIS LINE
bash scripts/audit-log.sh 5

EXPECTED OUTPUT — THE LAST ROWS

time (UTC)	caller	tool	ok	ms	args
<date> <time>	<your email>	lookup_stock	ok	<n>	{"book_id": "PRC-KYC-003"}
<date> <time>	<your email>	reserve_book	ok	<n>	{"book_id": "PRC-KYC-003", "copies": 1}

The reservation is in the log with your email and the arguments as sent. The record of a reservation used to be a line on your screen; now it is a line the server wrote, in Cloud Logging, that no client can edit.

- Fence it on the client side, whatever the server exposes. `INCLUDE_TOOLS` writes `includeTools` on the `bookshelf` entry — `lab-parts/part-d.settings.json` is the fragment, printed in Appendix A.

COMMAND — PASTE THIS LINE
`INCLUDE_TOOLS=lookup_stock,search_bookshelf bash scripts/gemini-config.sh`

EXPECTED OUTPUT

```
wrote /home/<you>/.gemini/settings.json
  agent-search: https://discoveryengine.googleapis.com/mcp
  bookshelf: https://bookshelf-mcp-<...>.a.run.app/mcp  includeTools=['lookup_stock', 'search_bookshelf']
Start Gemini CLI with: gemini  then type /mcp to see both servers and their tools.
```

COMMAND — PASTE THIS LINE
gemini

IN GEMINI CLI
/mcp

Two tools again under `bookshelf`. Ask it to reserve and it has nothing to call, exactly as in Part B — but this time the server still publishes three. Prove it from the terminal (`/quit` first):

COMMAND — PASTE THIS LINE
`python scripts/mcp-call.py --url $BOOKSHELF_MCP_URL/mcp --list`

EXAMPLE — THE TOOLS/LIST LINES

```
tools/list: 3 tools ttlMs=300000 cacheScope=public
- lookup_stock {"readOnlyHint": true}
- search_bookshelf {"readOnlyHint": true}
- reserve_book {"readOnlyHint": false, "destructiveHint": false, "idempotentHint": false}
```

Two fences, two owners. `EXPOSE_RESERVE` on the server is the system owner's: what exists, for every client. `includeTools` in `settings.json` is the client owner's: what this client will use, whatever exists. Neither can be undone by a prompt on the other side.

✓ Check yourself

`--token none`: refused — a 403 from Cloud Run, or the server's 401: `no valid Google ID token` on a local run. With your token: `tools/list: 2 tools`.

`audit-log.sh 20`: your earlier calls with your email in the `caller` column.

After `EXPOSE_RESERVE=on`: `tools/list: 3 tools`, `reserve_book` with `readOnlyHint: false`; the reservation from Gemini CLI is in the log with your email and its arguments.

After `INCLUDE_TOOLS: /mcp` shows two tools; `mcp-call.py --list` still shows three.

You can say which fence each owner holds, and why neither is a prompt.

Troubleshooting

Symptom	Cause	Fix
Step 1 with your token: <code>http: 403</code>	<code>roles/run.invoker</code> on <code>bookshelf-mcp</code> is missing for your account	The one-line grant in Part B's troubleshooting
<code>error: 401</code> with your token	The token expired or did not verify at the server	Run again — <code>--token auto</code> fetches a fresh one; in Gemini CLI, <code>gemini-config.sh</code> again
<code>no tool_call entries yet</code>	Cloud Logging has not received the lines yet, or the filter found nothing	Wait a few seconds and rerun; check <code>gcloud config get-value project</code>
<code>gcloud run services update</code> is refused	Your account cannot deploy revisions	The same roles as <code>deploy.sh</code> ; ask the project owner
<code>/mcp</code> still shows two tools after <code>EXPOSE_RESERVE=on</code>	Gemini CLI read the list at start, or the token in <code>settings.json</code> expired	<code>bash scripts/gemini-config.sh</code> , then start <code>gemini</code> again
The reservation is refused with <code>HTTP 409</code>	The library is low	<code>python agent.py --reset</code> in the agent kit

Part E — The agent through MCP

Goal

Put the tools behind the agent's own loop, then hold two servers in one client. With `MCP=on` the agent reads its tools from the server's `tools/list` instead of `tools.json`, and its third line says so; a reserve request finds no tool in the published list. Then, in Gemini CLI, both `agent-search` and `bookshelf` are configured at once — one client, two owners' capability, two identities. Optionally, put the approval card in front of the server's `reserve_book`.

Steps

1. Move to the agent kit and run it with `MCP=on`. `MCP=on` makes `agent.py` send `tools/list` to `MCP_URL` at start and run every call there through `tools/call`, with your ID token (`MCP_AUTH=on`, the default); `agent.env` and its `TOOLS=on` still apply.

COMMAND — PASTE THIS LINE

```
cd ../regai-d2-agent-kit
```

COMMAND — PASTE THIS LINE

```
MCP=on MCP_URL=$BOOKSHELF_MCP_URL/mcp python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
tools from: https://bookshelf-mcp-<...>.a.run.app/mcp (tools/list, 2 tools, ttlMs=300000, cacheScope=public,
  protocol 2026-07-28, token sent)
message: How many copies of PRC-KYC-003 are left?
→ model turn 1 tools=[lookup_stock, search_bookshelf] mode=AUTO grounding=off
→ model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙️ run lookup_stock({"book_id": "PRC-KYC-003"}) → {"book_id": "PRC-KYC-003", "title": "Customer
  Identification and Verification Procedure", "copies_available": 3, "copies_total": 3}
→ function_response lookup_stock {"book_id": "PRC-KYC-003", "title": "Customer Identification and
  Verification Procedure", "copies_available": 3, "copies_total": 3}
→ model turn 2 tools=[lookup_stock, search_bookshelf] mode=AUTO grounding=off
→ model text: <the answer – the title and how many of 3 copies are available>
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
```

The third line is new: `tools from: https://bookshelf-mcp-<...>.a.run.app/mcp (tools/list, 2 tools, ttlMs=300000, cacheScope=public, protocol 2026-07-28, token sent).tools.json` was never read. From there the printout is the loop's usual: `→ model` with two tool names, `← model` `function_call`, `⚙️ run lookup_stock(...)` `→ {...}` — except that the run happened on the server, with your identity, and the loop cannot tell the difference.

2. Ask the agent to reserve.

COMMAND — PASTE THIS LINE

```
MCP=on MCP_URL=$BOOKSHELF_MCP_URL/mcp python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"
```

EXAMPLE

```
caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
tools from: https://bookshelf-mcp-<...>.a.run.app/mcp (tools/list, 2 tools, ttlMs=300000, cacheScope=public,
  protocol 2026-07-28, token sent)
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf] mode=AUTO grounding=off
→ model text: <the model saying it cannot reserve – no tool for it>
steps=1 calls=0 tokens=<n> cost=$<n> elapsed=<n>s
```

`tools=[lookup_stock, search_bookshelf]` on the `→ model` line; no `function_call`; `steps=1` `calls=0`. The agent kit's `tools.json` still has `reserve_book` in it — and it does not matter, because the list came from the server. (If you left `EXPOSE_RESERVE=on` from Part D, the tool is in the list and the reservation runs; turn it off, or use the approval step below.)

- Two servers, one client. Back in this kit's folder, write both servers into Gemini CLI's settings (this is what `gemini-config.sh` has been doing), start it, and list them.

COMMAND — PASTE THIS LINE

```
cd ../regai-d4-mcp-kit && bash scripts/gemini-config.sh
```

COMMAND — PASTE THIS LINE

gemin

IN GEMINI CLI

/mcp

`/mcp` shows both servers connected at once: `agent-search`, reached with your Google credentials, and `bookshelf`, reached with your ID token — all their tools listed together. Ask a documents question and a stock question, one after the other:

IN GEMINI CLI

Using the bookshelf-app engine, which form must a cardholder sign before a dispute can go to chargeback?

IN GEMINI CLI

How many copies of PRC-KYC-003 are left?

Each goes to the server that has the right tool, and each call carries the identity that server expects — Google credentials to one, your ID token to the other. Which server Gemini chooses for the documents question is live (it may use `agent-search` or `bookshelf's search_bookshelf`); note it. Capability from two owners, composed in one agent, with no glue between them — because both speak the same `tools/list` and `tools/call`. Composition does not merge identities or trust: each server verifies its own caller and writes its own audit line. `/quit`.

4. Optional, if there is time — the approval card in front of a tool that lives on a server. In the agent kit, `APPROVE=ask` puts the card in front of `reserve_book` because the server's annotation says `readOnlyHint: false`; answer n. (Needs `EXPOSE_RESERVE=on` from Part D so the tool is in the list.)

COMMAND — PASTE THIS LINE

```
cd ../regai-d2-agent-kit
```

COMMAND — PASTE THIS LINE

EXAMPLE

```

caller: daniel (Daniel Okafor, Dispute Officer, clearance confidential)
model: gemini-2.5-flash (Vertex AI, global)
tools from: https://bookshelf-mcp-<...>.a.run.app/mcp (tools/list, 3 tools, ttlMs=300000, cacheScope=public,
protocol 2026-07-28, token sent)
message: Reserve one copy of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model function_call reserve_book({"book_id": "PRC-KYC-003", "copies": 1})
┌ approval needed _____
│ action    reserve_book
│ book      PRC-KYC-003 – Customer Identification and Verification Procedure
│ copies    1 (1 available now, 0 after)
│ caller    daniel – Daniel Okafor, Dispute Officer, clearance confidential
└ approve? [y/n]
approval: n
⚙️ run reserve_book({"book_id": "PRC-KYC-003", "copies": 1}) → blocked: declined by approver
→ function_response reserve_book {"error": "declined by approver"}
→ model turn 2 tools=[lookup_stock, search_bookshelf, reserve_book] mode=AUTO grounding=off
→ model text: <the model reporting that the reservation was declined>
steps=2 calls=0 tokens=<n> cost=$<n> elapsed=<n>s

```

The card is unchanged: action, the book with its title, copies with the stock before and after, the caller. What is new is where the fact that a gate is needed came from: not `tools.json`, which was never read, but the server's

`readOnlyHint: false`. Declined, `blocked: declined by approver`, `calls=0` — and the server's log has no `reserve_book` line for it, because the call never left the loop.

✓ Check yourself

`agent.py MCP=on`: the `tools from:` line with your URL, 2 `tools`, `token sent`; the lookup ran as  `run lookup_stock`; the reserve request ended with `calls=0`.

`/mcp` showed `agent-search` and `bookshelf` connected at the same time, with all their tools.

A documents question and a stock question were each answered by the right server — you noted which server the documents question used.

(Optional) `APPROVE=ask` printed the card in front of `reserve_book`; declining left no `reserve_book` line in the server's log.

Troubleshooting

Symptom	Cause	Fix
<code>tools from: ...</code> <code>tools/list failed: 401</code> in the agent	No valid ID token reached the server	<code>gcloud auth login</code> , run again. <code>MCP_AUTH=off</code> is only for a local server started with <code>AUTH=off</code>
The reserve request runs instead of finding no tool	<code>EXPOSE_RESERVE=on</code> is still set on the service	<code>gcloud run services update bookshelf-mcp --region asia-south1 --update-env-vars EXPOSE_RESERVE=off</code> , or use the approval step
Only one server shows in <code>/mcp</code>	The other's token expired, or its service is down	<code>bash scripts/gemini-config.sh</code> for a fresh token; check <code>bookshelf-mcp</code> is deployed
The optional step runs the reservation without a card	<code>APPROVE</code> was not typed in front of the command	Type <code>APPROVE=ask</code> before <code>MCP=on ...</code> , on the same line
<code>HTTP 409</code> on the reservation	The library is low	<code>python agent.py --reset</code> in the agent kit

Appendix A — settings.json, final (after Part D)

The `mcpServers` section of `~/ .gemini/settings.json` as `gemini-config.sh` leaves it at the end of Part D, with placeholders where the script fills in your project id, your service URL and a fresh ID token. This is `lab-parts/part-d/settings.json`. `agent-search` uses your own Google credentials through Gemini CLI's `google_credentials` provider; `bookshelf` carries an ID token in the `Authorization` header, which lasts about an hour — rerun the script when Gemini CLI reports a 401. `includeTools` is the client-side fence; `trust: false` makes Gemini CLI ask before each call.

KIT FILE — LAB-PARTS/PART-D.SETTINGS.JSON

```
{
  "mcpServers": {
    "agent-search": {
      "httpUrl": "https://discoveryengine.googleapis.com/mcp",
      "authProviderType": "google_credentials",
      "oauth": {
        "scopes": ["https://www.googleapis.com/auth/cloud-platform"]
      },
      "headers": {
        "X-Goog-User-Project": "<your-project-id>"
      },
      "timeout": 60000
    },
    "bookshelf": {
      "httpUrl": "https://bookshelf-mcp-<hash>-el.a.run.app/mcp",
      "headers": {
        "Authorization": "Bearer <output of: gcloud auth print-identity-token>"
      },
      "includeTools": ["lookup_stock", "search_bookshelf"],
      "timeout": 30000,
      "trust": false
    }
  }
}
```

Field	Meaning
httpUrl	The server's Streamable HTTP endpoint, <code>.../mcp</code>
authProviderType · oauth.scopes	<code>google_credentials</code> : Gemini CLI presents your Google credentials with these scopes (Agent Search)
headers	Sent on every request: <code>X-Goog-User-Project</code> for the Agent Search server; <code>Authorization: Bearer <ID token></code> for ours
includeTools / excludeTools	The client-side fence: which of the server's tools this client uses
trust	<code>false</code> : ask before each tool call
timeout	Milliseconds to wait for the server

Appendix B — The audit line

server.py prints one JSON line per tools/call to stdout, and one per refused request. Cloud Logging stores each as a structured entry (jsonPayload), which is what audit-log.sh filters on: resource.labels.service_name="bookshelf-mcp" and jsonPayload.event="tool_call". The server writes it, not the client — a client can lie in its own log; it cannot write this one.

EXAMPLE — A SUCCESSFUL CALL, A TOOL ERROR, AND A REFUSED REQUEST (LOCAL RUN)

```
{
  "event": "tool_call",
  "tool": "lookup_stock",
  "args": {
    "book_id": "PRC-KYC-003"
  },
  "caller": "<your email>",
  "ok": true,
  "ms": 1
},
{
  "event": "tool_call",
  "tool": "lookup_stock",
  "args": {
    "book_id": 5
  },
  "caller": "anonymous",
  "ok": false,
  "ms": 2,
  "error": "Error executing tool lookup_stock: 1 validation error for lookup_stockArguments\nbook_id\n  Input should be a valid string [type=string_type, input_value=5, input_type=int]\n  For further information v"
},
{
  "event": "tool_call",
  "tool": null,
  "args": null,
  "caller": "anonymous",
  "ok": false,
  "ms": 0,
  "error": "401: no valid Google ID token (no bearer token)",
  "method": "server/discover"
}
```

Field	Meaning
event	Always tool_call
tool	The tool called — or, on a refused request, the Mcp-Name header the client sent, if any
args	The arguments, as sent
caller	The email claim of the verified ID token, or anonymous when there was no valid token
ok	true unless the request was refused, the tool raised, or its result carried an error key
ms	Milliseconds the call took
error	Present when not ok: the reason, cut to 200 characters
method	On a refused request only: the Mcp-Method header a 2026-07-28 client sends (e.g. server/discover)

audit-log.sh N prints the last N entries oldest first: time, caller, tool (or (method) for a refusal), ok/FAIL, ms, args, and the error text when there is one. The columns are cut to fit; the JSON entry has the full values.

Appendix C — The 2026-07-28 revision on one page

The current revision of the Model Context Protocol specification, and where each item shows up today. The Python SDK the kit uses (`mcp 2.2.0`) implements it; the server is `MCPServer` served as a Streamable HTTP app with `stateless_http=True`.

Area	2026-07-28	Where you saw it
transport	Streamable HTTP: one POST per request to <code>.../mcp</code> ; HTTP+SSE deprecated; <code>Mcp-Method</code> and <code>Mcp-Name</code> headers on every POST	every <code>mcp-call.py</code> run; the <code>method</code> field on a refused request
stateless core	no <code>initialize</code> handshake, no <code>Mcp-Session-Id</code> ; every request carries <code>_meta</code> with the protocol version and client capabilities; list results the same for every connection; state as explicit server-minted handles	<code>server/discover: versions=['2026-07-28']; stateless_http=True</code> in <code>server.py</code>
<code>server/discover</code>	every server must implement it: supported versions, capabilities, identity	the first three lines of every listing
primitives	tools (model), resources (client), prompts (user); each with its own <code>*/list</code> and read/get calls	Part B (tools), Part C (<code>resources/list</code> , <code>resources/read</code> , <code>prompts/list</code> , <code>prompts/get</code>)
cacheable responses	<code>ttlMs</code> and <code>cacheScope</code> (<code>public</code> / <code>private</code>) required on <code>tools/list</code> , <code>prompts/list</code> , <code>resources/list</code> , <code>resources/read</code> ; tools in a deterministic order	<code>tools/list: 2 tools</code> <code>ttlMs=300000</code> <code>cacheScope=public;</code> <code>resources/read ...</code> <code>ttlMs=300000</code> ; the agent's <code>tools from:</code> line
extensions	an <code>extensions</code> field on client and server capabilities; tasks moved out of the core into <code>io.modelcontextprotocol/tasks</code>	none declared by our server
results	<code>resultType</code> required on every result; <code>input_required</code> replaces server-initiated requests such as elicitation	<code>structuredContent</code> and <code>isError</code> in <code>mcp-call.py --call</code>
authorization	clients validate <code>iss</code> against the recorded issuer (RFC 9207); credentials keyed by issuer, never reused across servers; Client ID Metadata Documents replace Dynamic Client Registration; <code>application_type</code> required	Gemini CLI's <code>google_credentials</code> flow for Agent Search
ours, concretely	a Google ID token per caller; Cloud Run IAM checks it, then <code>server.py</code> checks signature, issuer <code>accounts.google.com</code> , audience (the service URL) and expiry, and logs the <code>email</code>	Part D: refused without, listed with, your email in the log
deprecated	Roots, Sampling, Logging; HTTP+SSE; Dynamic Client Registration	—

Appendix D — The callers, and who is who (callers.json)

The three callers and filter strings, in `bookshelf-mcp/callers.json` — plus an `emails` map. The server never lets the model name the caller: it maps the verified Google account email to a caller through `emails`, and an email that is not listed is `priya`. The trainer fills the map before Part B; ask which entry is yours.

id	Name and role	Clearance	Filter string
priya	Priya Nair — Customer Contact agent	internal	<code>status: ANY("approved") AND classification: ANY("internal")</code>
daniel	Daniel Okafor — Dispute Officer	confidential	<code>status: ANY("approved") AND classification: ANY("internal","confidential")</code>
mlro	Sarah Whitmore — Deputy MLRO	restricted	<code>status: ANY("approved") AND classification: ANY("internal","confidential","restricted")</code>

KIT FILE — CALLERS.JSON, THE EMAILS MAP AS SHIPPED (THE TRAINER ADDS THE CLASS)

```
"emails": {
  "trainer@example.com": "mlro"
}
```

`search_bookshelf` sends the filter string of the caller the email maps to. That choice is made by the server from the token; `--user` only decides what the model is told about the person it is talking to.

Appendix E — The switches

Where	Switch	Effect
server	<code>AUTH=on/off</code>	verify the caller's Google ID token; <code>off</code> for local runs (caller <code>anonymous</code>)
server	<code>EXPOSE_RESERVE=on/off</code>	register <code>reserve_book</code> ; <code>off</code> withholds it (the deployed default). Part D sets it with <code>gcloud run services update</code>
server	<code>SERVICE_URL</code>	the audience an ID token must name (set by <code>deploy.sh</code>)
server	<code>FAKE_API=1</code>	in-process catalogue instead of <code>bookshelf-api</code> (offline runs only)
agent	<code>MCP=on/off</code>	tools from the server's <code>tools/list</code> , calls through <code>tools/call</code> ; <code>tools.json</code> not read
agent	<code>MCP_URL</code>	the endpoint, <code>https://.../mcp</code>
agent	<code>MCP_AUTH=on/off</code>	send your ID token (<code>gcloud auth print-identity-token</code>); <code>off</code> only for a local server
Gemini CLI	<code>includeTools / excludeTools</code>	the client-side fence, in <code>settings.json</code> ; <code>INCLUDE_TOOLS=...</code> on <code>gemini-config.sh</code> writes it

Everything the agent kit already does still works with the patched files: `MCP=off` (the default) reads `tools.json` as before, and `bash scripts/selftest.sh` there passes. This kit has its own `scripts/selftest.sh`, which runs the server, the client and the patched agent on this machine with no project and no network; `MEASURED=out.txt` keeps every printout.